

Android 第六章第一节教学讲义（数据存储入门版）

各位同学，今天咱们开始学习 Android 中的 "数据存储" 知识。想象一下，你用 APP 登录时勾选了 "记住密码"，下次打开 APP 不用重新输入；或者你设置了 APP 的主题颜色，关闭 APP 再打开依然是你设置的颜色 —— 这些都是因为数据被存储起来了。这节课咱们就来学习 4 种常用的数据存储方式，重点掌握 **SharedPreferences** 的用法，并用它实现 "记住密码" 功能。

一、先搞懂：为什么需要数据存储？

咱们之前写的 APP，数据都是 "临时" 的 —— 比如在 EditText 里输入的内容，关闭 APP 再打开就没了；登录时输入的账号密码，每次打开都要重新输。这显然不方便，所以需要把数据 "存起来"，让它在 APP 关闭后依然存在。

Android 提供了 4 种常用的数据存储方式，就像 4 种不同的 "存钱罐"，各有各的用途：

1. **SharedPreferences**：轻量级 "小钱包"，适合存简单的键值对（比如开关状态、账号密码）；
2. **SQLite**："存折本"，适合存大量结构化数据（比如通讯录、聊天记录）；
3. **存储卡**："大保险柜"，适合存文件（比如图片、视频、文档）；
4. **Application**："临时口袋"，适合存 APP 运行期间需要共享的数据（关闭 APP 就消失）。

今天咱们重点讲最常用的 **SharedPreferences**，其他方式后续课程会详细说。

二、SharedPreferences：轻量级数据存储

1. 什么是 SharedPreferences？

简单说，它是一种用 **键值对**（key-value）形式存储数据的方式，就像一本 "小字典"：每个数据都有一个名字（key）和对应的值（value），比如 "username" 对应 "admin"，"isRemember" 对应 "true"。

2. 特点和使用场合

- **轻量级**：存不了大量数据，适合存简单的配置信息（比如是否记住密码、主题设置、用户偏好）；

- **只能存简单类型**：支持字符串（String）、布尔值（boolean）、整数（int）、浮点数（float）、长整数（long）；
- **存储位置**：数据存在 APP 的私有目录里（/data/data/应用包名/shared_prefs/），其他 APP 看不到，安全！

3. 核心用法：存数据→取数据→改数据

步骤 1：获取 SharedPreferences 对象

就像打开 "小字典" 准备读写，需要指定一个文件名（比如 "user_info"）：

```
// 获取SharedPreferences对象，文件名是"user_info"
// MODE_PRIVATE：私有模式，只有当前APP能访问
SharedPreferences sp = getSharedPreferences("user_info", MODE_PRIVATE);
```

步骤 2：存数据（需要用 Editor 编辑器）

存数据就像在字典里写新词条，必须通过Editor对象操作：

```
// 1. 获取编辑器（相当于拿笔准备写）
SharedPreferences.Editor editor = sp.edit();
// 2. 存数据（key-value形式）
editor.putString("username", "admin"); // 存字符串：账号
editor.putString("password", "123456"); // 存字符串：密码
editor.putBoolean("isRemember", true); // 存布尔值：是否记住密码
// 3. 提交保存（必须调用，否则存不进去！）
editor.apply(); // 推荐用apply()，异步保存，不卡界面
// 也可以用editor.commit()，同步保存，适合需要立即知道结果的场景
```

步骤 3：取数据

从字典里查词条，根据 key 获取对应的值，需要指定默认值（如果没找到 key 就返回默认值）：

```
// 取账号（默认值是空字符串）
String username = sp.getString("username", "");
```

```
// 取密码（默认值是空字符串）
String password = sp.getString("password", "");
// 取是否记住密码（默认值是false）
boolean isRemember = sp.getBoolean("isRemember", false);
```

步骤 4：修改或删除数据

- 修改：用 `editor.putXxx()` 重新设置同一个 key 的值，再 `apply()`；
- 删除单个数据： `editor.remove("key") + apply()`；
- 清空所有数据： `editor.clear() + apply()`。

三、实战：用 SharedPreferences 实现 "记住密码" 功能

咱们基于上节课的登录界面，添加 "记住密码" 复选框，实现勾选后下次登录自动填充账号密码。

1. 先修改登录界面布局（加复选框）

在登录按钮上方添加一个 CheckBox：

```
<!-- 记住密码复选框 -->
<CheckBox
    android:id="@+id/cb_remember"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="记住密码"
    android:layout_marginBottom="15dp"/>
```

2. 登录时判断是否需要保存密码

在登录逻辑中，根据 CheckBox 的状态决定是否用 SharedPreferences 保存账号密码：

```
public class LoginActivity extends AppCompatActivity {
    private EditText etUsername;
    private EditText etPassword;
    private CheckBox cbRemember;
```

```
private SharedPreferences sp;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);
    // 初始化控件
    etUsername = findViewById(R.id.et_username);
    etPassword = findViewById(R.id.et_password);
    cbRemember = findViewById(R.id.cb_remember);
    Button btnLogin = findViewById(R.id.btn_login);
    // 获取SharedPreferences对象（文件名"user_info"）
    sp = getSharedPreferences("user_info", MODE_PRIVATE);
    // 页面打开时，自动填充已保存的账号密码
    loadSavedUserInfo();
    // 登录按钮点击事件
    btnLogin.setOnClickListener(v -> login());
}
// 加载已保存的账号密码
private void loadSavedUserInfo() {
    // 从SharedPreferences中取数据
    String username = sp.getString("username", "");
    String password = sp.getString("password", "");
    boolean isRemember = sp.getBoolean("isRemember", false);
    // 如果之前勾选了记住密码，就填充到输入框
    if (isRemember) {
        etUsername.setText(username);
        etPassword.setText(password);
        cbRemember.setChecked(true); // 复选框保持勾选状态
    }
}
// 登录方法
private void login() {
    String username = etUsername.getText().toString().trim();
    String password = etPassword.getText().toString().trim();
    // 简单验证（实际项目中需联网验证）
    if (username.isEmpty() || password.isEmpty()) {
        Toast.makeText(this, "账号或密码不能为空", Toast.LENGTH_SHORT).show();
        return;
    }
    // 假设验证通过
    Toast.makeText(this, "登录成功", Toast.LENGTH_SHORT).show();
}
```

```
// 判断是否需要记住密码
if (cbRemember.isChecked()) {
    // 勾选了：保存账号密码
    SharedPreferences.Editor editor = sp.edit();
    editor.putString("username", username);
    editor.putString("password", password);
    editor.putBoolean("isRemember", true);
    editor.apply(); // 保存
} else {
    // 没勾选：清除已保存的密码（可选）
    SharedPreferences.Editor editor = sp.edit();
    editor.remove("password"); // 只删密码，保留账号
    editor.apply();
}
// 登录成功后跳转到首页
finish();
}
```

3. 效果演示

- 第一次登录：输入账号 "admin"、密码 "123456"，勾选 "记住密码"，点击登录；
- 关闭 APP 再打开：输入框会自动填充账号和密码，复选框保持勾选状态；
- 取消勾选：下次打开 APP，账号保留，密码为空。

四、其他数据存储方式简介（后续详解）

1. SQLite：

- 关系型数据库，适合存大量结构化数据（比如一个 APP 的用户列表，每条用户信息包含 ID、姓名、年龄等）；
- 支持增删改查（CRUD）操作，需要写 SQL 语句或用 Android 提供的辅助类。

2. 存储卡：

- 分为 "内部存储"（APP 私有，卸载后删除）和 "外部存储"（公共区域，比如 SD 卡，卸载后可能保留）；
- 适合存图片、视频、文档等大文件，需要申请存储权限。

3. Application:

- 整个 APP 只有一个 Application 实例，在 APP 运行期间一直存在；
- 适合存临时共享数据（比如用户登录后的 token，多个页面都需要用到），但关闭 APP 后数据会消失。

总结

这节课咱们重点学习了：

1. **数据存储的意义**：让数据在 APP 关闭后依然保留，提升用户体验；

2. SharedPreferences:

- 轻量级，存键值对，适合简单配置（如记住密码）；
- 用法：`getSharedPreferences()`获取对象→`editor.putXxx()`存数据→`editor.apply()`保存→`sp.getXxx()`取数据；

3. **实战案例**：用 SharedPreferences 实现 "记住密码"，核心是登录时保存、打开时读取。

课后作业：给登录界面再加一个 "自动登录" 复选框，实现勾选后下次打开 APP 直接登录（不需要点击登录按钮）。提示：可以在 `loadSavedUserInfo()` 方法中判断如果是自动登录，就直接执行登录逻辑～有问题随时问哦！

（注：文档部分内容可能由 AI 生成）